

慕思意图识别模型微调步骤总结

意图识别模型微调结果

1) 微调场景介绍

整个模型微调包含3个微调场景，形成一个***意图识别→路由分发→专业响应***的三层架构：

层级	场景	模型规模	作用
第一层：意图路由	Task3 意图判别	4B	识别用户意图，分发到对应处理模块
第二层：知识问答	Task1 睡眠顾问QA	14B	处理睡眠知识、身体健康、心理健康等咨询
第二层：报告解读	Task2 睡眠报告解读	14B	处理睡眠报告的专业分析与解读

现状痛点：之前用 Qwen3-14B 基座 + 提示词做所有事，上下文 token 长度达 5000-10000，推理速度慢、格式不稳、资源重，意图识别作为链路的首个环节，每次交互都要调用，如果用 14B 大模型做分类，就是极大的资源浪费而且慢。

意图识别模型核心设计思想：小模型做分类，大模型做生成。 意图识别是一个分类任务，计算量小但需要快速响应，所以用4B小模型；睡眠问答和报告解读是生成式任务，需要深度理解和生成长文本，所以用14B大模型。

意图识别的核心角色

Task3意图识别模型是整个系统的***入口守门员***，其判别结果直接决定了用户请求的路由走向：

代码块

1 用户输入 → [意图识别模型(4B)] → 判别意图类别

2

3

(14B)

4

型(14B)

5

—— 环境控制(4) → 直接调用IoT设备控制接口

—— 睡眠报告(5) → 路由到Task2报告解读模型

—— 知识顾问(0,1,2,3) → 路由到Task1睡眠QA模型

—— 其他问题(6) → 通用对话兜底处理

这意味着：意图识别的准确率直接决定了整个系统的可用性。如果意图判错，用户说"帮我调高床垫"却被路由到了知识问答，体验将极差。因此，意图识别虽然是"最小"的模型，却是**整个系统最关键的模块**。

2) 微调全流程技术路线概览

整个模型微调过程包括：定场景->定目标->定边界->选基座->备数据->选方法->配参数->训练->评估->迭代->部署

定场景	定目标	定边界	选基座	备数据	选方法	配参数
睡眠意图分类	F1>85%、4B约束、实时推理	7类标签、4→路由、优先级	19+模型对比、筛选到4B	种子数据+合成增强、3维扩展	3种架构×3格式对比	LoRA/全参差量化

3) 微调结果

最终实际结果：

- F1达到 **90.48%**（同尺寸Qwen3-4B原始模型82.01%，飙升+8.47%）
- 超越 Qwen3-14B 基座（84.45%），实现 **SOTA性能反超**
- 格式错误率降至 **0%**

微调过程的11个步骤具体逻辑

步骤一：定场景

慕思将意图识别定位为**"睡眠领域智能对话系统的意图路由网关"**。

具体场景定义：

- 业务场景：** 用户通过语音/文字与慕思智能硬件交互，系统需要自动识别用户意图并路由到正确的处理模块
- 技术场景：** 多分类（7类）文本分类任务，支持多轮对话上下文理解

从7分类到4分类的场景简化，最初意图识别设计了7个标签：产品售后(0)、睡眠知识(1)、身体健康(2)、心理健康(3)、环境控制(4)、睡眠报告(5)、其他问题(6)

最终上线时合并为4类：**知识顾问**（合并了产品售后+睡眠知识+身体健康+心理健康）、**睡眠报告**、**环境控制**、**其他问题**，为什么合并？因为知识顾问的4个子类别最终都路由到同一个下游处理节点（睡眠顾问问答模型），细分它们在业务上没有额外价值，反而增加分类难度。

之前用 Qwen3-14B 基座 + 提示词做所有事，上下文 token 长度达 5000-10000，推理速度慢、格式不稳、资源重，意图识别是**判别式任务**（输出是标签），需要的是快速、准确、格式统一，意图识别要优化的是**延迟和吞吐**（每次都跑，要快）。

步骤二：定目标

核心目标：以小博大，降本增效

目标维度	具体指标	为什么设这个目标
降本增效	以 Qwen3-4B（40 亿参数）替代 Qwen3-14B（140 亿参数）	参数量减少70%+，显存占用从14B降到4B，支持更高并发
精准分类	F1 > 89%， Accuracy > 89%	覆盖7大核心意图（后合并为4类），解决相似类别混淆问题
规范输出	格式错误率降至 0%	通过SFT强制输出统一Label，确保下游工作流100%可执行
极速响应	推理速度提升3 倍+	无需Prompt，减少输入Token，通过减少模型层数降低延迟

步骤三：定边界

(1) 类别边界的精确定义

每个意图标签都有明确的核心定义和"硬边界"标识：

标签	核心定义与边界	边界类型
产品售后(0)	硬件与App：坏了、连不上、怎么用、保养、退换货	硬边界
睡眠知识(1)	睡这类事：失眠、做梦、打鼾、睡姿、生物钟、早起	核心域
身体健康(2)	吃/动/病：饮食、运动、身体疼痛、特殊人群日常护理	—
心理健康(3)	情/心：焦虑、抑郁、压力、情绪宣泄	需特殊Prompt
环境控制(4)	动设备：抬高、放平、加热、按摩、关灯	硬边界
睡眠报告(5)	查数据：多少分、时长、心率、昨晚怎么样	硬边界
其他问题(6)	闲聊/噪音：天气、算术、竞品、无关话题	防误触

(2) 意图混淆边界的详细枚举

这是慕思核心的边界工程——穷举了所有可能的混淆情况：

实际意图	混淆意图	判断规则	举例
知识顾问 ↔ 睡眠报告	提到"报告、评分"等词，但其实是在问"共性知识"	区分"我的数据" vs "一般性知识"	"深睡比例一般多少算正常" → 知识顾问
知识顾问 ↔ 环境控制	句子带"调高、调低"等词，但本质是在问"建议/原理"	是否有明确的"做事"指令	"睡前把灯调暗会不会更好入睡" → 知识顾问
睡眠报告 ↔ 知识顾问	在讨论"我自己的数据"，但后半句是泛化问题	主体是"我的数据"还是"一般性问题"	"评分60分，一般人睡眠评分多少才算正常" → 睡眠报告
环境控制 ↔ 知识顾问	有明确控制命令，但拼在一句话里像在问建议	是否涉及设备动作	"帮我把床头抬高一点，最近老打鼾" → 环境控制
其他问题 ↔ 环境控制	语句里有"调、开、关、温度"等词，但在说习惯/描述	是否是当下命令	"我每次睡觉前都要把空调调到26度才觉得舒服" → 其他问题

(3) 多意图优先级规则

当用户输入中出现多个意图时，设计了明确的优先级排序：

环境控制 > 睡眠报告 > 知识顾问 > 其他问题

每个优先级背后的业务逻辑：

- **环境控制最优先**：涉及设备动作，有体感风险甚至安全问题。用户的主观预期是"你应该先帮我执行"，而不是先聊天讲知识。业务原则：**动作优先于解释**
- **睡眠报告次优先**：报告是个人数据+服务关键卖点，误判成知识顾问会让用户不满。业务原则：只要涉及"我的报告/我的数据"，就优先认为是报告交互
- **知识顾问再次**：大部分咨询都落在这里，如果既不是执行动作、也不是围绕个人报告，大概率就是知识顾问。被误判成其他（OTHER）危害相对小
- **其他问题兜底**：闲聊、无关问题、噪音输入。一旦有任何迹象指向前三类，就不要落在OTHER。业务原则：**宁可错判到某个业务意图，也尽量不要把有诉求的问法扔进OTHER**

(4) 数据层面的边界控制——种子生成策略

为确保数据覆盖面足够广，从3个维度生成训练数据种子：

维度	种子类别	目的
语义类种子	睡眠知识主题、身体健康、心理健康、慕思产品实体、产品功能模块、产品问题类型、报告指标种子、设备类型、控制指令等	覆盖每个意图的内容空间
表达方式类种子	句式类型（陈述/疑问/祈使/选择）、语气/礼貌程度、口语化/错别字/缩写/表情、方言/网络表达、信息缺省/指代缺省	覆盖用户的表达多样性
场景类种子	用户身份/角色、使用场景、设备状态/生命周期、多意图组合结构、噪音/干扰信息种子	覆盖使用场景的多样性

(5) 数据量边界

每个标签的建议数据量也是精心设计的：

标签	建议数据量	为什么
睡眠知识	2,500	核心域，话题最广，需要最多数据覆盖
产品售后	1,500	硬边界，需要精准区分各种售后场景
环境控制	1,500	硬边界，设备控制指令多样
睡眠报告	1,500	硬边界，涉及各种数据查询和解读请求
其他问题	1,800	防误触，需要大量负样本让模型学会"不要乱归类"
身体健康	1,200	话题相对集中
心理健康	1,000	话题最聚焦，但需特殊Prompt

总计约 **10,926条** 训练数据。

步骤四：选基座

1.1 选型范围：三大架构家族、19+模型配置

慕思项目在意图识别（Task3）场景中，系统性地对比了**三大模型架构家族**：

架构类型	具体模型	参数量级
Encoder（BERT系列）	bert-base-chinese、chinese-macbert-base、chinese-roberta-wwm-ext、deberta-v3-base、chinese-lert-base	~110M
Decoder-only（Qwen3 LLM）	Qwen3-0.6B、Qwen3-1.7B、Qwen3-4B-Instruct-2507	0.6B~4B
Embedding模型	Qwen3-Embedding-0.6B、Qwen3-Embedding-4B	0.6B~4B

1.2 选型实验设计

项目不仅对比了模型架构，还交叉对比了**3种数据输入格式**：

数据格式	适用模型	格式特点
single_sequence	BERT + Qwen3 LLM	只输入当前用户问题，无上下文
context_query	BERT	历史对话作为context，当前问题作为query，双输入
chat_template	Qwen3 LLM + Qwen3 Embedding	完整的多轮对话消息列表

这意味着实际评测的模型配置数 = 5个BERT × 2种格式 + 3个LLM × 2种格式 + 2个Embedding × 1种格式 = **18种配置**，加上Qwen3-14B baseline共**19+种**。

1.3 最终选型结果

从评测文档中19个模型的F1得分来看：

- **最终胜出：Qwen3-4B-Instruct + SFT微调**，F1=90.48%
- 对比基线：Qwen3-14B（不微调，纯prompt） F1=84.45%
- BERT系列最佳：chinese-lert-base + context_query，F1约85%+
- **4B模型微调后超越了14B基线模型6个百分点**

这验证了项目的核心假设：**小模型+领域微调 > 大模型+通用能力**。

步骤五：备数据

1.1 标签体系设计：从7类到实际映射

核心标签映射（来自 `label_config.py`）：

代码块

```
1 LABEL_MAP = {
2     0: "产品售后",
3     1: "睡眠知识",
4     2: "身体健康",
5     3: "心理健康",
6     4: "环境控制",
7     5: "睡眠报告",
8     6: "其他问题"
9 }
```

为什么是7个类别？ 这是经过业务梳理后的结果：

- 原始业务场景可能有更细的类别，但太多类别会导致分类困难
- 7个类别的设计遵循了**互斥且完整（MECE）原则**：每个用户意图有且只有一个归属类别，所有可能的意图都能被覆盖（"其他问题"作为兜底）
- 从标签映射文档可以看到，项目做了大量的**边界定义**工作，例如区分"产品售后"和"环境控制"的关键在于用户是"咨询"还是"指令"

意图优先级排序：环境控制 > 睡眠报告 > 知识顾问 > 其他问题。这解决了多意图歧义问题——当一个输入可能属于多个类别时，按优先级决定归属。

1.2 数据采集与种子数据策略

慕思项目的数据备制采用了**"种子数据 + 大模型合成增强"**的策略：

Step 1：采集真实种子数据

- 从慕思实际用户交互日志中提取真实对话
- 人工标注初始种子数据集
- 每个标签类别确保有基础覆盖量

Step 2：三维度种子扩展策略（来自数据需求文档）

种子数据从三个维度进行扩展：

维度	说明	示例
语义维度	同一意图的不同表达方式	"床怎么调高"→"怎么升高床垫"→"床的高度能改吗"
表达风格维度	口语化/书面化/方言化	"帮我调高床"→"请将床调至较高位置"→"床弄高点"
场景维度	不同使用场景下的表达	睡前/起床后/故障时/首次使用时

Step 3：大模型合成增强

- 使用大模型（如Qwen3-14B）基于种子数据生成更多样本
- 每个种子从三个维度分别扩展，实现数据量的指数级增长
- 生成后经过人工审核和去重

1.3 数据分布与规模

从数据需求文档中可以看到各标签的数据量规划：

标签	训练样本量	说明
产品售后	较多	高频场景，需要充足样本
睡眠知识	较多	知识面广，需要多样性
身体健康	中等	与睡眠知识存在边界模糊
心理健康	中等	需要敏感问题的覆盖
环境控制	较多	高优先级，不能误判
睡眠报告	中等	与睡眠知识存在边界模糊
其他问题	中等	兜底类别，需要多样性

测试集固定为**696条样本**，所有19+模型配置在同一测试集上评测，确保可比性。

1.4 数据的工程化处理

从代码结构 `data_project/task3/` 可以看出数据准备有完整的工程化流程：

- 数据合成增强代码
- 去重转换代码（避免重复样本导致过拟合）
- 统计分析代码（检查各类别数据分布是否均衡）
- 数据集格式为JSONL文件
- 训练时自动拆分10%作为验证集（`val_size: 0.1`）
- 最大序列长度：BERT为512，LLM和Embedding为1024
- 支持数据打包（packing），但对话任务建议关闭

步骤六：选方法

1.1 慕思的方法选择全景图

慕思在"选方法"这一步，做了**三个层面的方法选择**，每个层面都有明确的对比和决策逻辑：

决策层面	候选方案	最终选择
架构路线	BERT Encoder vs Qwen3 Decoder-only vs Qwen3-Embedding	三条路线并行实验，最终Qwen3-4B胜出
微调策略	全参数微调 vs LoRA微调 vs QLoRA(4bit/8bit量化+LoRA)	BERT用全参数，LLM和Embedding用LoRA
数据格式方案	single_sequence vs context_query vs chat_template	每种架构配2种格式，交叉实验

1.2 训练框架选择

任务	框架	原因
Task3（意图分类）	自研 <code>multi_classify_factory</code>	需要支持3种架构的分类头改造，LlamaFactory不支持
Task1（睡眠QA）	LlamaFactory	标准SFT任务，LlamaFactory完美支持

自研框架的技术设计：

核心设计是**统一训练入口 + 架构分支**：

- 统一的命令行参数定义（ `ModelArguments` , `DataTrainingArguments` ）
- 统一的数据加载流程（JSONL → HuggingFace Dataset）
- 按 `model_architecture_type` 分支的模型加载、数据预处理
- 统一的 `Trainer` API做训练
- 统一的 `compute_metrics` 做评估

这种设计让19+种模型配置可以通过**一个脚本+不同参数**来运行，极大降低了实验管理的复杂度。

步骤七：配参数

1.1 慕思的参数配置全景对比

慕思为不同架构配置了**差异化的超参数**，每个参数的选择都有明确的技术逻辑，分类模型训练（Task3）三套参数对比：

参数	BERT系列	Qwen3 LLM系列	Qwen3 Embedding系列	差异原因
Epochs	10	10	10	统一，便于对比
Learning Rate	2.00E-05	2.00E-04	1.00E-04	微调策略不同
Batch Size	64	8	16	显存约束不同
Max Length	512	1024	1024	位置编码约束
Gradient Accumulation	4	4	4	统一，放大有效 batch
Warmup Ratio	0.1	0.1	0.1	统一
Weight Decay	0.01	0.01	0.01	统一
LoRA Rank	—	32	16	模型特性不同
LoRA Alpha	—	64	32	alpha=2×rank
LoRA Dropout	—	0.05	0.05	统一
LoRA Target	—	q,k,v	q,k,v,o	Embedding需要更多投影
Precision	fp16	fp16	fp16	统一
分布式	2 GPU (torchrun)	2 GPU (torchrun)	2 GPU (torchrun)	统一
LR Scheduler	cosine	cosine	cosine	统一

步骤八：训练

1.1 训练体系全景图

慕思的训练体系由**两套并行的训练流水线**组成，分别服务于不同的技术路线：

代码块

1

流水线A：分类头微调（自研 multi_classify_factory）

2

├── 19+种模型配置（BERT×5 + LLM×3 + Embedding×2，各配2种数据格式）

3

├── 统一入口：trainer_utils.py（1490行）

4

├── 统一测试集：696条（4种格式版本）

5

└── 训练数据：10926条

6

7

流水线B：SFT微调（LlamaFactory）

8

├── Qwen3-4B SFT v1（rank=32）和 v2（rank=16）

9

└── Qwen3-14B 基线对比

步骤九：评估

1.1 评估体系全景图

慕思构建了一套**四层架构**的评估系统，具备工程级的健壮性和可扩展性：

代码块

1	
2	配置层 (Config Layer)
3	test.yaml → 定义模型、数据集、API地址、model_type等
4	
5	解析层 (Adapter Layer)
6	model_adapter.py → 按model_type构建输入消息
7	response_parser.py → 按model_type解析模型输出
8	
9	指标计算层 (Metrics Layer)
10	security_evaluator.py → F1/Acc/P/R + 混淆矩阵 + 分类报告
11	
12	报告生成层 (Report Layer)
13	eval_results.csv + metrics.csv + confusion_matrices.csv
14	+ class_report.csv + performance_summary
15	

1.2 评估数据与测试集

测试集规模和分布:

意图ID	意图名称	样本数	占比
0	产品售后	118	16.95%
1	睡眠知识	127	18.25%
2	身体健康	73	10.49%
3	心理健康	91	13.07%
4	环境控制	98	14.08%
5	睡眠报告	103	14.80%
6	其他问题	86	12.36%
总计		696	100%

4种格式的测试集：

格式	文件名	适用模型	数据结构
single_sequence	scenere_test1.json	BERT/Qwen single_sequence	{"id", "text", "label"}
context_query	scenere_test2.json	BERT context_query	{"id", "context", "query", "label"}
chat_template	scenere_test3.json	Qwen chat_template	{"id", "messages", "label"}
chat_template/SFT	scenere_test4.json	Qwen SFT	{"id", "messages", "label"} (含 history/query标签)

为什么696条？为什么这个分布？

- 696条确保每个类别至少有73条样本（最少的是"身体健康"），足以计算有统计意义的per-class指标
- 分布**不完全均衡**（最大127/最小73 = 1.74:1），这是有意设计——反映真实业务中各意图的自然频率
- 如果强制均衡（每类100条），评估结果会过于乐观，无法反映模型在真实分布下的表现

1.3 评估推理流程详解

慕思为不同模型类型设计了**6种适配器**，统一了输入构建和参数配置：

代码块

```
1  _ADAPTER_REGISTRY = {
2      'default': DefaultAdapter,           # 标准OpenAI模型
3      'single_sequence': BERT_Single_sequence, # BERT单序列
4      'context_query': BERT_context_query,     # BERT双序列
5      'qwen_sft': QwenSFTdAdapter,           # Qwen SFT模型
6      'chat_template': Qwen_chat,           # Qwen原生chat_template
7      'qwen_base': QwenSFTdAdapter         # Qwen基线模型
8  }
```

每种适配器的差异化实现：

适配器	build_messages输入	build_messages输出	API参数特殊点
BERT_Single_sequence	text字符串	<code>[{"role": "user", "content": "text"}]</code>	stream=False
BERT_context_query	原始行字典	<code>{"messages": [], "extra_body": {"context": str, "query": str}}</code>	max_tokens=512
QwenSFTdAdapter	text(JSON字符串)	解析JSON为messages列表, 含system prompt	temperature=0, repetition_penalty=1.2
Qwen_chat	text(JSON字符串)	直接解析为messages列表	同上

关键设计：Encoder模型 vs Decoder模型的API调用方式不同

代码块

```
1  # security_evaluator.py中的分支逻辑
2  if set_type == 'decoder':
3      final_params = {**api_params, "messages": messages}  # 标准OpenAI格式
4  else:  # encoder
5      final_params = {**api_params, **messages}  # 展开dict, 包含extra_body
```

Encoder模型（BERT）需要通过 `extra_body` 传递 `context` 和 `query` 字段（非标准OpenAI格式），因此需要特殊处理。

不同模型的输出格式不同，需要对应的解析器：

代码块

```
1  _PARSER_REGISTRY = {
2      'single_sequence': parse_bert_single_response,  # 正则匹配 (ID: 3)
3      'qwen_sft': parse_qwen3_SFT_response,          # JSON解析 {"id": 3}
4      'context_query': parse_bert_single_response,
5      'chat_template': parse_qwen3_SFT_response,
6      'qwen_base': parse_base                        # 文本→标签映射
7  }
```

步骤十：迭代

1.1 迭代全景图

慕思的迭代过程横跨**3个维度**，涉及**20+模型变体**和**多个数据/参数版本**：

代码块

```
1  维度1：架构迭代（横向探索）
2      BERT×5模型×2格式 → Qwen3 LLM×3规模×2格式 → Qwen3 Embedding×2规模 → Qwen3-4B
   SFT
3      ↓ 同期并行实验
4
5  维度2：数据迭代（纵向深入）
6      数据v1(1216版) → 评估 → 分析weak point → 数据v2/v3调整 → 重新训练
7
8  维度3：参数迭代（精细调优）
9      SFT v1(rank=32, 12月22日) → 评估 → SFT v2(rank=16, 12月25日) → 评估
```

1.2 聚焦最优

慕思的架构迭代遵循了一个*****"漏斗式"筛选****的过程：

代码块

```
1  第一轮：全面铺开（19+配置同时实验）
2  |—— BERT家族：5模型 × 2格式 = 10配置
3  |—— Qwen3 LLM：3模型 × 2格式 = 6配置（含NPU版本）
4  |—— Qwen3 Embedding：2模型 × 1-2格式 = 2-3配置
5  |—— 统一评估：696条测试集，F1 macro排序
6
7  第二轮：聚焦胜出架构
8  |—— 结论：single_sequence/context_query格式 >> chat_template格式
9  |—— 结论：Qwen3-4B-Instruct-2507-single_sequence 排名第一
10 |—— 决定：在Qwen3-4B上做SFT深度优化
11
12 第三轮：SFT深度迭代
13 |—— v1(rank=32) → 评估F1=81.96%
14 |—— v2(rank=16) → 评估F1=83.68%
15 |—— 持续迭代优化...
```

从模型记录PDF和评测文档可以还原出迭代时间线：

时间节点	迭代内容	关键发现
12月16日	准备训练数据 (intent_classify_task_sft_1216)，10926条	数据集v1完成
12月18日	训练环境Docker搭建，开始BERT系列训练	训练文档记录了NPU环境的aten::empty_strided错误
12月22日	SFT v1完成 (qwen3_4b_task3_gpu_v1_1222)	rank=32, F1=81.96%
12月23日	所有分类头模型训练完成(所有模型名含"1223")	19种分类头配置全部评估完毕
12月25日	SFT v2完成 (qwen3_4b_task3_gpu_v2_1225)	rank降至16, F1提升到83.68%
后续	持续数据+参数迭代	最终SFT模型达到F1=90.48%

步骤十一：部署

1.1 部署架构全景图

慕思构建了一套完整的模型服务化体系，覆盖从LoRA合并到API服务的全链路：

代码块

1		
2	LoRA合并层	
3	merge_lora.py → 基座模型+LoRA适配器 → 合并后的完整模型	
4		
5	模型服务层	
6	intent_recognition_api.py (FastAPI)	
7	├─ 19+模型配置注册表(MODEL_CONFIGS)	
8	├─ 模型热加载/卸载机制	
9	├─ OpenAI兼容API (/v1/chat/completions)	
10	└─ 3种输入格式的推理路径	
11		
12	硬件适配层	
13	├─ GPU部署 (NVIDIA A100/V100)	
14	└─ NPU部署 (华为昇腾, Docker容器化)	

合并脚本（merge_lora.py）的核心逻辑：

代码块

```

1  # 1. 自动检测模型架构
2  model_name_lower = model_path.lower()
3  if 'qwen3' in model_name_lower:
4      arch = 'qwen3'
5  elif 'qwen2' in model_name_lower:
6      arch = 'qwen2'
7  else:
8      arch = 'auto'
9
10 # 2. 加载基座模型
11 base_model =
    AutoModelForSequenceClassification.from_pretrained(base_model_path)
12
13 # 3. 加载LoRA适配器
14 model = PeftModel.from_pretrained(base_model, adapter_path)
15
16 # 4. 合并权重
17 merged_model = model.merge_and_unload()
18
19 # 5. 保存合并后的模型 (safetensors格式)
20 merged_model.save_pretrained(output_path, safe_serialization=True)

```

Task1的合并配置（qwen3_14b_lora_sft_merge.yaml）：

代码块

```

1  model_name_or_path: ./models/Qwen3-14B
2  adapter_name_or_path: ./swanlab_outputs/sft/qwen3_14b_task1_lora_gpu_v6
3  template: qwen3
4  export_dir: ./outputs/qwen3_14b_task1_gpu_v6_1225
5  export_size: 5          # 分片大小5GB
6  export_device: auto
7  export_legacy_format: false

```

合并 vs 不合并的对比：

维度	不合并（运行时加载LoRA）	合并后部署
推理速度	慢5-10%（额外矩阵乘法）	标准速度
推理代码	需要PEFT库	标准 HuggingFace/ vLLM
文件管理	base_model + adapter两份文件	单份合并文件
兼容性	受PEFT版本限制	任何推理框架都能加载
可回溯性	可以随时拆离LoRA	无法拆离

1.2 FastAPI服务

intent_recognition_api.py的核心架构：

代码块

```
1  # FastAPI应用
2  app = FastAPI(title="Intent Recognition API")
3
4  # 19+模型配置注册表
5  MODEL_CONFIGS = {
6      # BERT系列（10种配置）
7      "bert-base-chinese-single_sequence-classifier": {
8          "model_path": "...", "arch_type": "bert_classifier", "format":
"single_sequence"
9      },
10     "bert-base-chinese-context_query-classifier": {
11         "model_path": "...", "arch_type": "bert_classifier", "format":
"context_query"
12     },
13     "chinese-macbert-base-context_query-classifier": {...},
14     "chinese-macbert-base-single_sequence-classifier": {...},
15     "chinese-roberta-wwm-ext-context_query-classifier-GPU_V1": {...},
16     # ... 更多BERT配置
17
18     # Qwen3 LLM系列（6种配置）
19     "Qwen3-0.6B-single_sequence-classifier": {...},
20     "Qwen3-0.6B-chat_template-classifier": {...},
21     "Qwen3-4B-Instruct-2507-single_sequence-classifier": {...},
22     # ...
```

```

23
24     # Qwen3 Embedding系列 (2种配置)
25     "Qwen3-Embedding-0.6B-chat_template_classifier": {...},
26     # ...
27
28     # SFT模型
29     "qwen3_4b_task3_gpu_v1_1222": {...},
30     "qwen3_4b_task3_gpu_v2_1225": {...},
31 }

```

关键功能实现：

(1) 模型热加载/卸载机制

代码块

```

1  # 当前加载的模型
2  loaded_models = {} # {model_name: (model, tokenizer)}
3
4  @app.post("/v1/load_model")
5  async def load_model(model_name: str):
6      """将指定模型加载到GPU"""
7      config = MODEL_CONFIGS[model_name]
8      model =
9      AutoModelForSequenceClassification.from_pretrained(config["model_path"])
9      tokenizer = AutoTokenizer.from_pretrained(config["model_path"])
10     model.to("cuda").eval()
11     loaded_models[model_name] = (model, tokenizer)
12
13     @app.post("/v1/unload_model")
14     async def unload_model(model_name: str):
15         """从GPU卸载指定模型"""
16         del loaded_models[model_name]
17         torch.cuda.empty_cache() # 释放GPU显存

```

模型合并：

- LoRA权重与基座模型合并：使用 `merge_lora.py` 将LoRA adapter权重永久合并到基座模型中
- 合并配置（以Task1为例）：`base=Qwen3-14B + adapter=qwen3_14b_task1_lora_gpu_v6 → export=qwen3_14b_task1_gpu_v6_1225`
- 合并后模型无需运行时加载PEFT库，推理路径更简单

服务化部署：

- 使用 **FastAPI** 构建推理服务 (`intent_recognition_api.py`)

- 提供 **OpenAI兼容API** 接口（`/v1/chat/completions` 格式）
- 支持**模型热加载/卸载**：可以在不停服务的情况下切换模型
- 19+模型配置的按需加载：只将当前使用的模型加载到GPU，其他模型从显存中卸载

部署架构：

代码块

```
1  用户请求 → API Gateway → FastAPI服务 → GPU推理
2                                     |—— 模型加载管理器（热加载/卸载）
3                                     |—— 请求格式适配器（适配不同模型的输入格式）
4                                     |—— 结果解析器（统一输出格式）
```

产研资料：

<https://deepexi.yuque.com/pxb7n8/atnvko/mwd5sxep5t0bsgnm>

<https://gitlab.deepexi.com/modelhub/deepeximusifactory/-/tree/dev>